

Modulator głosu w czasie rzeczywistym

Cyfrowe Przetwarzanie Sygnałów

Autor: Marcin Drąg
Kontakt: github.com/Hasafah
Data: Sierpień 2025



Wszystkie ilustracje, schematy oraz ikony zawarte w pracy są autorstwa własnego i stanowią oryginalne opracowanie autora.

All graphics, diagrams, and icons in this thesis are the original work of the author. All rights reserved.

© Marcin Drąg

Jeśli interesuje Cię wyłącznie sposób uruchomienia aplikacji, przejdź bezpośrednio do podsekcji 3.1, gdzie opisano cały proces krok po kroku.

Spis treści

Spis treści	2
1 Wprowadzenie i skrót funkcjonalności	3
2 Interfejs użytkownika (UI)	3
2.1 Zakładka Effects	3
2.2 Zakładka Plots	5
2.3 Nagrywanie	6
3 Uruchamianie i parametry wykonania programu	7
3.1 Uruchamianie	7
3.2 Parametry	7
3.3 Organizacja wątków w programie oraz sposoby przetwarzania informacji	8
4 Architektura i przepływ sygnału	9
5 Opis algorytmów i efektów	10
5.1 Robot	10
5.2 Echo	11
5.3 Low Pitch i High Pitch	11
5.4 Radio	12
5.5 Flanger	12
5.6 Chorus	13
5.7 Time Stretch	14
5.8 Reverb	14
5.9 Bandpass	15
5.10 Online Classroom	16
5.11 Ring Modulation	17
5.12 Stutter	18
6 Wizualizacja sygnału	18
7 Obsługa - Podsumowanie i praktyczne wskazówki	19
8 Struktura projektu (pliki kluczowe) i rozwiązywanie problemów	19
Literatura	21

1 Wprowadzenie i skrót funkcjonalności

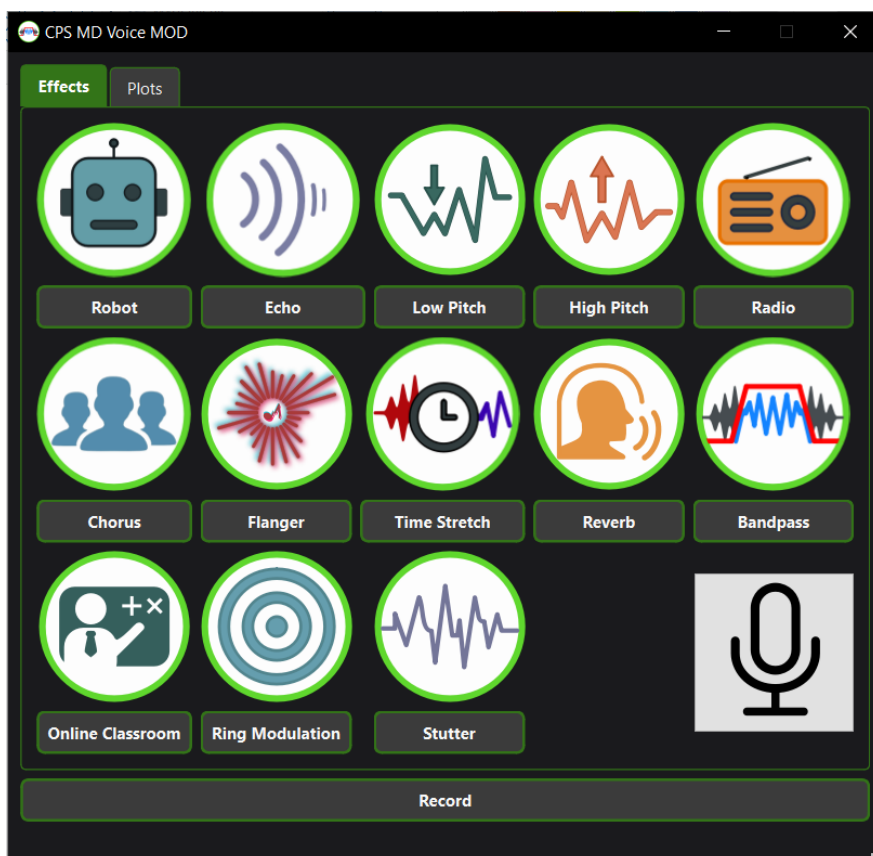
CPS MD Voice MOD to aplikacja czasu rzeczywistego do modyfikacji głosu i wizualizacji sygnału audio. Program przechwytuje dźwięk z domyślnego mikrofonu, przetwarza go wybranym efektem DSP i odtwarza na wyjściu audio. Umożliwia też podgląd przebiegu w czasie (waveform), spektrogramu oraz nagrywanie przetworzonego sygnału do WAV.

2 Interfejs użytkownika (UI)

Interfejs aplikacji **CPS MD Voice MOD** został zaprojektowany tak, aby użytkownik mógł intuicyjnie poruszać się pomiędzy dwoma głównymi funkcjonalnościami: wyborem efektów przetwarzania głosu oraz obserwacją wizualizacji sygnału.

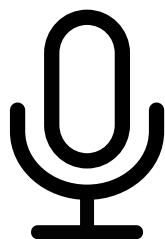
2.1 Zakładka Effects

Po uruchomieniu programu pojawia się okno główne, z otwartą zakładką *Effects*, które pełni rolę pulpitu sterującego. Tutaj użytkownik wybiera jeden z trzynastu dostępnych efektów modulacji głosu. Każdy efekt przedstawiony jest graficznie w formie ikony, która ułatwia szybkie rozpoznanie charakteru działania danego modułu (np. robot dla efektu robota, trzy osoby dla efektu chóru). Poniżej każdej ikony znajduje się przycisk z nazwą efektu. Wciśnięcie przycisku powoduje aktywację efektu, a przycisk zmienia swój wygląd, aby wizualnie podkreślić jego aktywny stan. Ponowne naciśnięcie wyłącza efekt i przywraca neutralny wygląd. W danym momencie aktywny może być tylko jeden efekt, co gwarantuje przejrzystość obsługi i brak niepożądanych nakładek między różnymi algorytmami DSP.



Rysunek 1: Interfejs aplikacji w zakładce Effects

W prawym dolnym rogu zakładki ***Effects*** umieszczono specjalny przycisk w formie ikony mikrofonu. Pozwala on na szybkie przełączanie pomiędzy stanem aktywnego wejścia audio a całkowitym wyciszeniem mikrofonu. Zmiana stanu przycisku jest natychmiast komunikowana zarówno poprzez ikonę (Rysunek 2 przedstawia aktywny mikrofon; Rysunek 3 - mikrofon wyciszony), jak i poprzez krótki komunikat pojawiający się na pasku statusu w dolnej części aplikacji. Dzięki temu użytkownik może w dowolnym momencie całkowicie odłączyć wejście bez konieczności sięgania do systemowych ustawień audio.



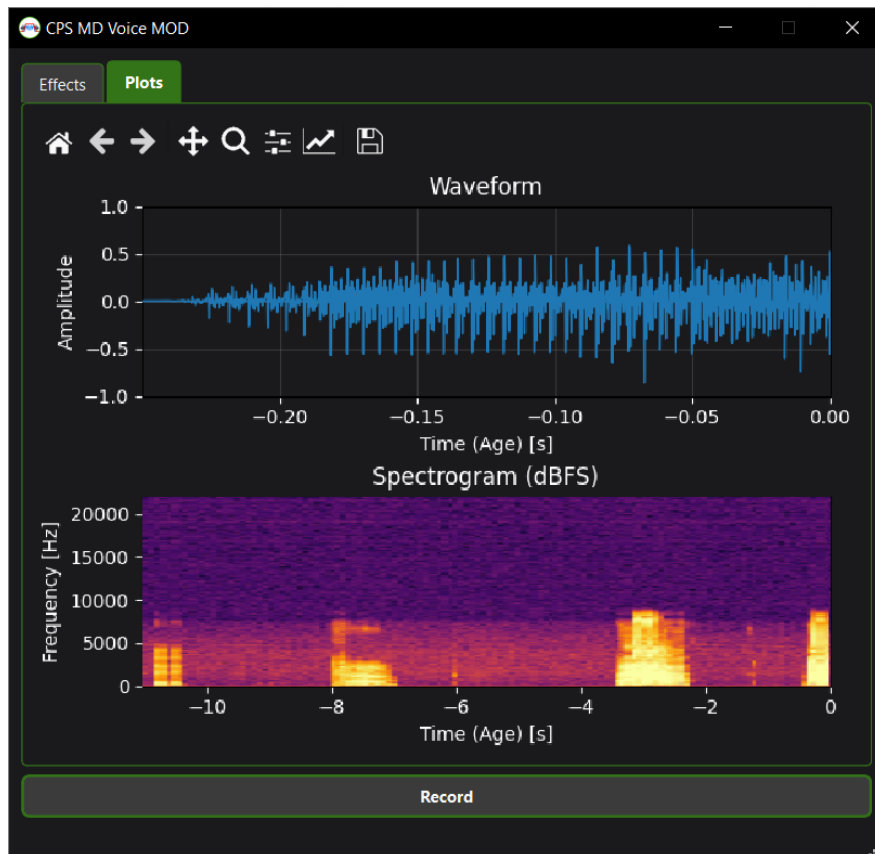
Rysunek 2: Ikona aktywnego mikrofonu



Rysunek 3: Ikona wyciszonego mikrofonu

2.2 Zakładka Plots

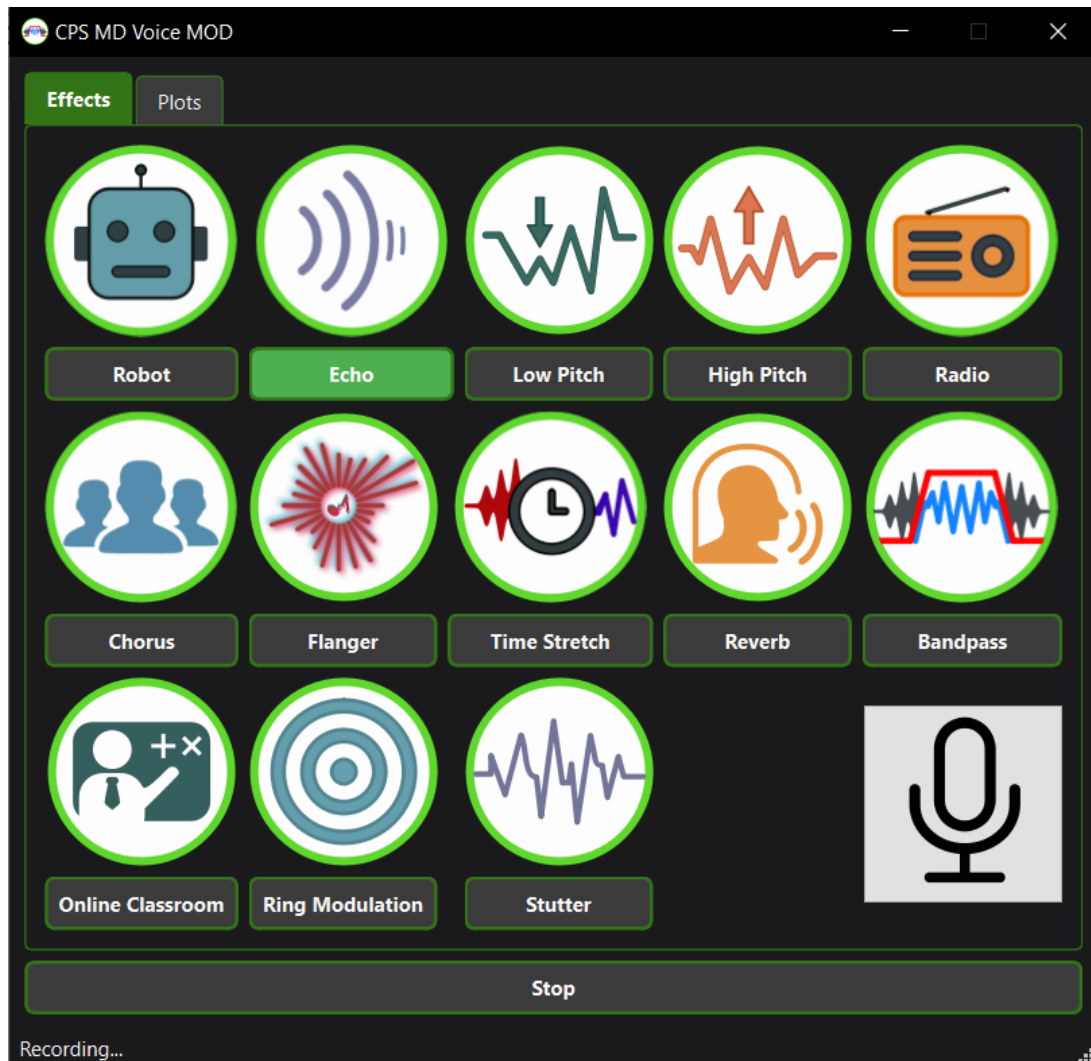
Drugą zakładką jest **Plots**, w której użytkownik ma możliwość obserwacji dwóch reprezentacji sygnału dźwiękowego. W górnej części znajduje się wykres przebiegu czasowego (waveform), który pokazuje kształt sygnału w ostatnim przedziale czasu, z osią czasu przesuniętą w stronę wartości ujemnych: od -0.25 sekundy do chwili obecnej oznaczonej jako zero. Dzięki temu możliwe jest śledzenie, jak wyglądają kolejne porcje próbek wchodzące do algorytmu DSP. Poniżej umieszczono spektrogram w skali decybelowej, będący reprezentacją czasu i częstotliwości. Spektrogram przewija się w czasie, a oś pozioma oznaczona jest jako Time (Age), czyli czas względny cofający się od teraźniejszości do 12 sekund wstecz. Oś pionowa sięga aż do częstotliwości Nyquista, czyli połowy częstotliwości próbkowania ($22,05$ kHz). Obie wizualizacje są renderowane w czasie rzeczywistym z użyciem mechanizmu blitowania, co pozwala na płynne odświeżanie obrazu bez zauważalnych spowolnień w działaniu programu. Nad wykresami umieszczono pasek narzędzi znany z Matplotlib, który umożliwia powiększanie, przesuwanie czy zapisywanie obrazu do pliku.



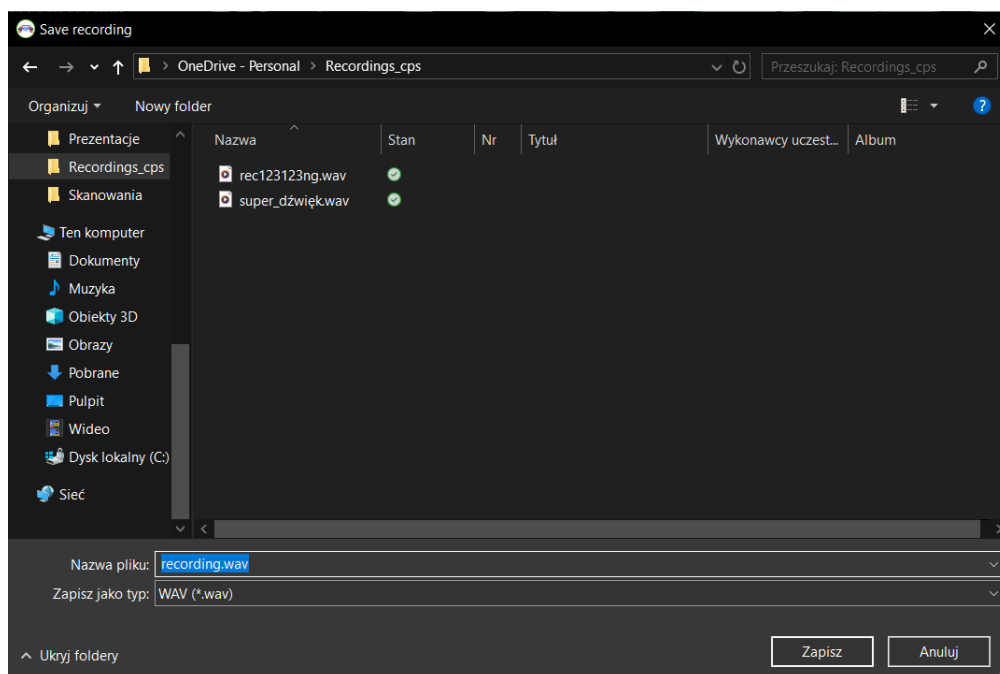
Rysunek 4: Interfejs aplikacji w zakładce Plots

2.3 Nagrywanie

Na dole okna, niezależnie od wybranej zakładki, znajduje się przycisk nagrywania, którego wygląd i opis dynamicznie zmieniają się w zależności od stanu — w trybie spoczynku oznaczony jest jako **Record**, natomiast podczas nagrywania zmienia się na Stop. Naciśnięcie **Stop** powoduje otwarcie okna dialogowego, w którym użytkownik może zapisać przetworzony dźwięk w formacie WAV. Rozwiązanie to umożliwia łatwe archiwizowanie nagrań bez konieczności stosowania zewnętrznych konwerterów.



Rysunek 5: Interfejs aplikacji w zakładce *Effects* podczas nagrywania dźwięku



Rysunek 6: Okno do zapisu nagrałego pliku

3 Uruchamianie i parametry wykonania programu

3.1 Uruchamianie

Aplikacja **CPS MD Voice MOD** dostarczana jest w postaci kompletnego zestawu plików projektu w Pythonie, zawierającego wszystkie źródła, arkusze stylów oraz pliki zasobów. Użytkownik, który chce po prostu uruchomić program, nie musi jednak wchodzić w szczegóły implementacyjne.

Aplikacja została przygotowana w taki sposób, aby jej uruchomienie nie wymagało od użytkownika żadnej specjalistycznej wiedzy czy dodatkowej konfiguracji. Do poprawnego działania wystarcza folder tworzony przez proces budowania przy pomocy PyInstaller. Znajduje się on w katalogu **dist** i nosi nazwę **CPSMDVoiceMOD**. Wewnątrz niego znajdują się dwa kluczowe elementy: plik wykonywalny **CPSMDVoiceMOD.exe** oraz katalog **_internal** zawierający wszystkie zaszyte biblioteki i zasoby. Aby uruchomić program, należy dwukrotnie kliknąć na plik **CPSMDVoiceMOD.exe**.

Dzięki takiemu rozwiązaniu aplikacja startuje w środowisku Windows bez konieczności ręcznej instalacji bibliotek i konfiguracji — wszystkie potrzebne komponenty (PySide6, PyAudio^[1], NumPy, SciPy, Matplotlib) są już dołączone do pakietu. Program automatycznie wykorzystuje domyślne urządzenia audio zdefiniowane w systemie, zwykle zintegrowany mikrofon i głośniki lub słuchawki. Nie ma konieczności wyboru urządzeń wewnątrz programu; proces inicjalizacji przebiega automatycznie, co upraszcza obsługę dla użytkownika.

Jeżeli plik wykonywalny **.exe** nie otwiera się w poprawny sposób, można skorzystać z alternatywnego sposobu uruchamiania aplikacji bezpośrednio z kodu źródłowego w Pythonie. Szczegółowa instrukcja krok po kroku znajduje się w sekcji 8.

3.2 Parametry

Domyślnym parametrem pracy systemu jest częstotliwość próbkowania 44,1 kHz, która stanowi standard w przetwarzaniu dźwięku i zapewnia pasmo przenoszenia do 22 kHz. Sygnał przechwytywany jest w postaci 16-bitowej, z jednym kanałem (mono). Strumień przetwarzany jest porcjami po 2048 próbek, co odpowiada

około 46 ms czasu rzeczywistego. Taka wielkość bufora stanowi kompromis między stabilnością przetwarzania a latencją. Im większy bufor, tym mniejsze ryzyko zakłóceń spowodowanych opóźnieniami w działaniu systemu operacyjnego, lecz równocześnie rośnie całkowite opóźnienie dźwięku. Wybrana wartość 2048 próbek pozwala zachować latencję na poziomie akceptowalnym dla zastosowań efektowych, gdzie dodatkowe dziesiątki milisekund opóźnienia nie stanowią istotnego problemu i są niemal niezauważalne.

3.3 Organizacja wątków w programie oraz sposoby przetwarzania informacji

Ważnym elementem jest sposób organizacji wątków i komunikacji między nimi. Główny wątek programu obsługuje wyłącznie interfejs graficzny, natomiast osobny wątek dedykowany jest przetwarzaniu audio. W tym drugim wątku uruchamiany jest strumień PyAudio w trybie callback. Oznacza to, że co każde 2048 próbek biblioteka PyAudio wywołuje zdefiniowaną funkcję, która odpowiedzialna jest za pobranie danych z mikrofonu, ich przetworzenie zgodnie z wybranym efektem DSP i odesłanie zmodyfikowanego sygnału na wyjście. Takie rozwiązanie gwarantuje nieprzerwane działanie w czasie rzeczywistym oraz minimalizuje ryzyko zawieszania się głównego wątku GUI.

Poza przetwarzaniem i odtwarzaniem sygnału w czasie rzeczywistym, aplikacja prowadzi także równoległą wizualizację. Aby nie obciążać nadmiernie CPU, nie każda ramka jest wysyłana do modułu rysującego wykresy. Zastosowano prostą strategię podpróbkiowania: co drugi blok trafia jako dane wejściowe do aktualizacji waveformu i spektrogramu. Pozwala to utrzymać płynność rysowania i responsywność programu, bez widocznego wpływu na dokładność prezentowanych informacji.

Oprócz głównego toru przetwarzania istotne są również kwestie nagrywania sygnału. Program umożliwia zapis tego, co aktualnie słyszymy na wyjściu, w formie pliku audio. Mechanizm ten działa w prosty sposób: kolejne porcje przetworzonych próbek gromadzone są w buforze, a po zakończeniu nagrywania łączone i zapisywane. Taki sposób zapisu informacji sprawia, że z nagranego dźwięku automatycznie wycinane są fragmenty, w których mikrofon był wyciszony. Domyślnym formatem jest WAV z parametrami identycznymi jak w torze odtwarzania (44,1 kHz, mono, 16-bit). W przypadku braku ffmpeg, aplikacja sama zadba o konwersję do WAV i odpowiednie poinformowanie użytkownika. Dzięki temu proces nagrywania jest odporny na brak zewnętrznych bibliotek.

4 Architektura i przepływ sygnału

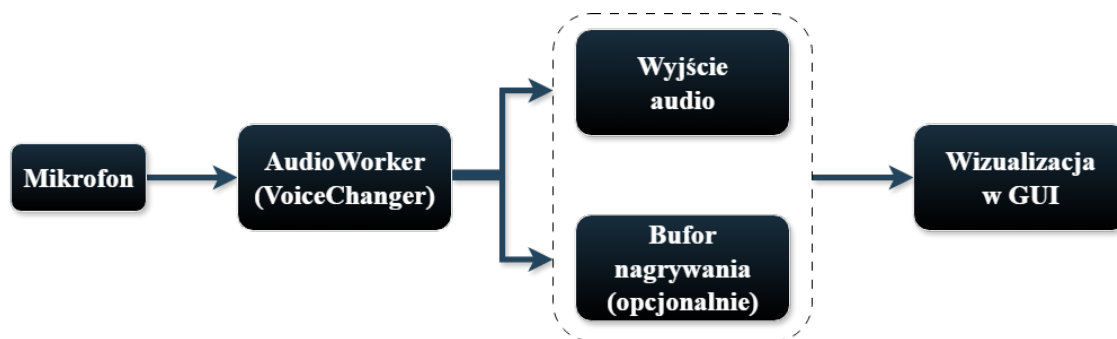
Wątek GUI w programie obsługuje interfejs użytkownika, zakładki, przyciski i wizualizacje, natomiast wątek audio odpowiada wyłącznie za przechwytywanie i przetwarzanie dźwięku w czasie rzeczywistym. Dzięki temu interfejs pozostaje responsywny, a strumień audio nie jest blokowany przez operacje graficzne.

Kluczową rolę pełni obiekt `AudioWorker`, który otwiera strumień `PyAudio`. Strumień pracuje w trybie callback, wywoływana jest funkcja, która pobiera dane z mikrofonu, przetwarza je w module `VoiceChanger` i odsyła na wyjście. W tym samym miejscu, gdy aktywne jest nagrywanie, bloki po efektach trafiają do bufora zapisu, a co kilka wywołań próbki w formacie `float32` wysyłane są do wątku GUI jako dane wizualizacyjne.

Klasa `VoiceChanger` implementuje wszystkie efekty DSP. W zależności od wyboru użytkownika blok wejściowy przechodzi np. przez filtrację IIR (efekt Radio, Bandpass), operacje na linii opóźniającej (Echo, Flanger, Chorus), konwolucję (Reverb) albo algorytmy w dziedzinie częstotliwości i granularne (Pitch Shift, Time Stretch). Bufory wewnętrzne utrzymują stan między kolejnymi wywołaniami callbacku, tak aby efekty zachowywały ciągłość w czasie.

Wizualizacja realizowana jest w wątku GUI. Co pewien interwał dane z callbacku przesyłane są do panelu Plots, gdzie zasilają bufor przebiegu czasowego i spektrogramu.

W momencie zamknięcia programu główny wątek wysyła sygnał do `AudioWorker`, aby zakończyć strumień, zwolnić zasoby `PyAudio` i bezpiecznie zamknąć aplikację. Całość przepływu sygnału można streścić następująco:



5 Opis algorytmów i efektów

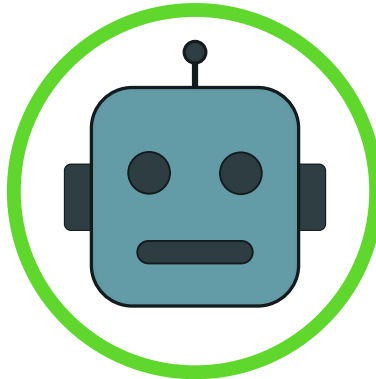
Najważniejszą częścią aplikacji jest moduł VoiceChanger, w którym zaimplementowano zestaw trzynastu efektów przetwarzania sygnału. Każdy z efektów realizuje różne efekty cyfrowego przetwarzania sygnałów: od prostych filtrów IIR, przez linie opóźniające i konwolucję, aż po operacje w dziedzinie częstotliwości i granularne metody zmiany czasu trwania sygnału.

5.1 Robot

Efekt “Robot” realizowany jest poprzez modulację amplitudy sygnału nośną w postaci fali prostokątnej o częstotliwości 30 Hz. Szczegółowa implementacja wygląda następująco:

```
t = np.arange(len(samples)) / self.rate
modulator = np.sign(np.sin(2 * np.pi * 30 * t))
return samples * modulator
```

Algorytm wykorzystuje funkcję `np.sign()` do przekształcenia sinusoidy w falę prostokątną o wartościach ± 1 . Mnożenie próbek audio przez ten modulator powoduje periodyczne odwrócenie fazy sygnału. W wyniku tego zanika wyraźna struktura harmoniczna mowy odpowiedzialna za wysokość tonu, a pozostaje poszarpana, sztuczna artykulacja. Celem efektu jest uzyskanie brzmienia kojarzonego z syntezatorami i “głosem robota”.



Rysunek 7: Ikona efektu “Robot”

5.2 Echo

Echo jest najprostszą realizacją efektu liniowego opóźnienia. Algorytm przechowuje w buforze fragment sygnału i dodaje go do bieżącej ramki po określonym czasie. W programie Opóźnienie wynosi 250 ms (11025 próbek przy 44.1 kHz), a opóźniona kopia jest tłumiona przez współczynnik 0.7. Powstaje wrażenie pojedynczego odbicia od ściany, a celem efektu jest wprowadzenie wrażenia przestrzeni i głębi dźwięku.



Rysunek 8: Ikona efektu ‘Echo’

5.3 Low Pitch i High Pitch

Zmiana wysokości głosu realizowana jest metodą prostego przeskalowania widma w dziedzinie częstotliwości. Najpierw sygnał jest mnożony przez okno Hanninga dla redukcji artefaktów spektralnych. Po wykonaniu FFT na ramce próbek, częstotliwości zostają skompresowane (dla Low Pitch, współczynnik 0.8) lub rozciągnięte (dla High Pitch, współczynnik 1.3), a następnie rekonstruowane za pomocą odwrotnej transformaty. Szczegółowa implementacja wygląda następująco:

```
# pitch_factor: Współczynnik zmiany tonacji (0.8 = niższy, 1.3 = wyższy)
def pitch_shift_fft(self, samples, pitch_factor):
    N = len(samples)
    window = np.hanning(N)
    fft_spectrum = np.fft.fft(samples * window) # Transformata FFT
    # Przesunięcie składowych częstotliwościowych
    indices = np.round(np.arange(0, N) * pitch_factor).astype(int)
    indices = indices[indices < N]
    shifted_spectrum = np.zeros(N, dtype=complex)
    shifted_spectrum[indices] = fft_spectrum[:len(indices)]
    # Odwrotna transformata FFT do dziedziny czasu
    shifted_signal = np.fft.ifft(shifted_spectrum).real
    return shifted_signal
```

Metoda ta jest szybka i intuicyjna, lecz wprowadza delikatne artefakty fazowe. Mimo ograniczeń dobrze ilustruje podstawowe podejście do pitch-shiftingu.



Rysunek 9: Ikona efektu “Low Pitch”



Rysunek 10: Ikona efektu “High Pitch”

5.4 Radio

Effekt radiowy to filtracja pasmowoprzepustowa zrealizowana filtrem IIR Butterwortha czwartego rzędu. Zakres przenoszenia ustawiony jest na 300 – 3400 Hz, co odpowiada charakterystyce telefonicznej. Efekt usuwa zarówno bardzo niskie, jak i bardzo wysokie częstotliwości, pozostawiając wąskie pasmo odpowiedzialne za zrozumiałość mowy. Jego celem jest symulacja brzmienia radia lub starego telefonu. Szczegółowa implementacja jest bardzo prosta:

```
b, a = butter(4, [300 / (self.rate/2), 3400 / (self.rate/2)], btype='band')
return lfilter(b, a, samples) * 0.8
```



Rysunek 11: Ikona efektu “Radio”

5.5 Flanger

Flanger implementuje zaawansowane modulowane opóźnienie z sprzężeniem zwrotnym i wykorzystuje Low Frequency Oscillator (LFO). Algorytm wykorzystuje:

- Bufor cykliczny do przechowywania historii sygnału
- LFO sinusoidalny o częstotliwości 0.25 Hz do modulacji opóźnienia
- Interpolację liniową między próbkami dla płynnej modulacji:

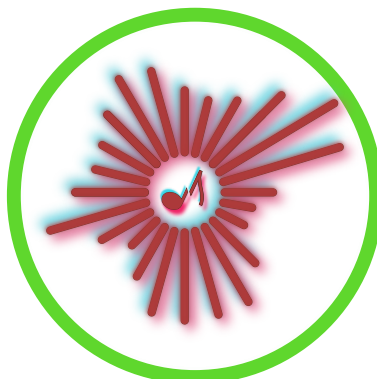
$$\text{delayed} = \text{buf}[\text{r0}] * (1 - \text{frac}) + \text{buf}[\text{r1}] * \text{frac}$$

- Sprzężenie zwrotne:

$$\text{buf}[\text{w}] = \text{samples}[\text{i}] + \text{feedback} * \text{delayed}$$

Opóźnienie zmienia się sinusoidalnie między 0.7 ms a 3.2 ms. Sprzężenie zwrotne ($\text{feedback} = 0.5$) powoduje, że opóźniona kopia jest ponownie wprowadzana do bufora, tworząc nieskończoną serię ech o malejącej amplitudzie. W dziedzinie częstotliwości powstają filtry grzebieniowe (comb filters) o zmiennej częstotliwości, co daje charakterystyczny efekt "przemiatania" przez spektrum.

Efekt jest szeroko stosowany w muzyce elektronicznej, nadając sygnałowi charakterystyczne "fale".



Rysunek 12: Ikona efektu "Flanger"

5.6 Chorus

Chorus wykorzystuje kilka niezależnych linii opóźniających o większych wartościach opóźnień (kilkanaście–kilkadziesiąt milisekund). Każda linia modulowana jest oddzielnym LFO o losowo dobranej fazie i nieznacznie różnej częstotliwości. Dzięki temu kolejne kopie sygnału nakładają się w czasie, powodując efekt wielogłosowości. Parametry typowe dla aplikacji to trzy głosy o opóźnieniach bazowych 18 ms, głębokości modulacji 12 ms i częstotliwości modulacji 0,8 Hz.



Rysunek 13: Ikona efektu "Chorus"

5.7 Time Stretch

Algorytm time-stretch realizuje metodę granularną. Sygnał dzielony jest na krótkie ziarna o długości 40 ms, które są wygładzane oknem Hanninga i następnie składane w nowej kolejności z innym krokiem czasowym. Parametr stretch określa stopień wydłużenia lub skrócenia sygnału, w implementacji ustawiony na 1.25, co oznacza spowolnienie o 25%. Ziarna są nakładane z 50% pokryciem, aby uniknąć artefaktów. Efekt pozwala zmienić tempo mowy bez modyfikacji jej wysokości. Sama implementacja granularnego time-stretchu jest stosunkowo złożona, jednak jej istotę można sprowadzić do wyznaczenia długości ziarna, kroków wejściowych i wyjściowych oraz okna wygładzającego. Fragment kodu ilustrujący ten mechanizm przedstawiono poniżej:

```
grain = int(max(16, grain_ms * 1e-3 * self.rate))
# Obliczenie przesunięcia między ziarnami wejściowymi
hop_in = max(8, int (grain * (1.0 - overlap) ))
# Obliczenie przesunięcia między ziarnami wyjściowymi
hop_out = max(8, int ( hop_in * stretch ) )
win = np.hanning(grain).astype(np.float32)
```



Rysunek 14: Ikona efektu ‘Time Stretch’

5.8 Reverb

Reverb symuluje pogłos pomieszczenia poprzez konwolucję sygnału z krótką odpowiedzią impulsową (IR). Odpowiedź ta zawiera kilka impulsów o różnym opóźnieniu i tłumieniu, co odwzorowuje wielokrotne odbicia dźwięku w przestrzeni. Operacja realizowana jest w dziedzinie częstotliwości przy użyciu algorytmu FFT, dzięki czemu jest wydajna obliczeniowo. Fragment kodu realizujący ten mechanizm wygląda następująco:

```
ir = np.zeros(8000)
ir[0] = 1.0
ir[500] = 0.5
ir[1500] = 0.35
ir[3000] = 0.25
ir[5000] = 0.15
ir[7000] = 0.08
reverb = fftconvolve(samples, ir, mode='full')[:len(samples)]
```

Efekt ma za zadanie symulować naturalne zjawisko akustyczne polegające na odbiciach dźwięku od różnych powierzchni w pomieszczeniu, takich jak ściany, sufit czy meble. Dzięki temu efektowi dźwięk zyskuje przestrzeń i głębię, co sprawia, że brzmi, jakby był odtwarzany w dużym pomieszczeniu, np. sali koncertowej, katedrze czy studiu o naturalnej akustyce.

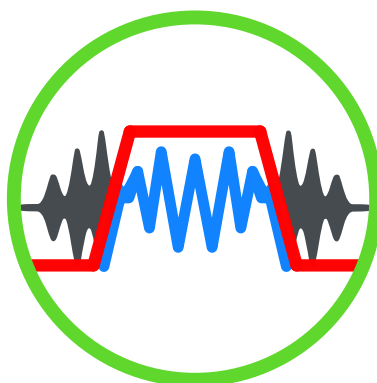


Rysunek 15: Ikona efektu “Reverb”

5.9 Bandpass

Bandpass wykorzystuje filtr pasmowoprzepustowy IIR Butterwortha czwartego rzędu. Domyślnie częstotliwość środkowa pasma wynosi 1200Hz, a dobroć filtru Q ustawiona jest na 1.0, co daje pasmo o szerokości 1200 Hz. Efekt pozwala uwypuklić wybrany fragment widma sygnału, nadając mu selektywne brzmienie. Kluczowe kroki implementacji wyglądają następująco:

```
bw = f0 / max(q, 1e-3)
# Obliczenie dolnej i górnej granicy pasma (z ograniczeniami)
low = max(30.0, f0 - bw/2.0) / (self.rate/2)
high = min(self.rate/2 - 100.0, f0 + bw/2.0) / (self.rate/2)
# Projektowanie filtra Butterwortha 4-go rzędu
b, a = butter(4, [low, high], btype='band', output='ba')
```



Rysunek 16: Ikona efektu “Bandpass”

5.10 Online Classroom

Efekt Online Classroom jest symulatorem zakłóceń w transmisji głosu przez Internet. Algorytm losowo wprowadza cisze (dropouty), przesunięcia czasowe, zanikanie głośności, trzaski, ograniczenie pasma czy dodanie szumu. Każde z tych zjawisk odzwierciedla typowe problemy związane z utratą pakietów czy niestabilnością sieci. Efekt jest przykładem wykorzystania prostych narzędzi DSP (filtry, szum, operacje okienkowe) do realistycznej symulacji.

Algorytm wykorzystuje generator liczb pseudolosowych do wprowadzania różnych typów zakłóceń z określonym prawdopodobieństwem:

```
chunk_len = len(samples)
# Losowe przerwy (cisza)
if np.random.rand() < 0.05:
    return np.zeros_like(samples)

# Opóźnienie z losowym czasem
elif np.random.rand() < 0.10:
    delay_ms = np.random.uniform(50, 200) # 50-200ms
    delay_samples = int(delay_ms * self.rate / 1000)
    delayed = np.concatenate((np.zeros(delay_samples), samples)[:chunk_len])
    return delayed * np.random.uniform(0.7, 0.9) # Losowe tłumienie

# Zanikanie dźwięku z losowym profilem
elif np.random.rand() < 0.15:
    fade_type = np.random.choice(['out', 'in_out'])
    if fade_type == 'out':
        fade = np.linspace(1, np.random.uniform(0.1, 0.4), chunk_len)
    else: # in_out
        mid_point = chunk_len // 2
        fade = np.concatenate((
            np.linspace(1, 0.3, mid_point),
            np.linspace(0.3, 1, chunk_len - mid_point)
        ))
    return samples * fade

# Dodanie szumu tła
elif np.random.rand() < 0.10:
    noise_level = np.random.uniform(0.05, 0.15)
    noise = noise_level * np.random.randn(chunk_len) * np.max(np.abs(samples))
    return samples + noise

# Losowe "trzaski"
elif np.random.rand() < 0.08:
    out = samples.copy()
    for _ in range(np.random.randint(1, 4)):
        pos = np.random.randint(0, chunk_len)
        duration = int(self.rate * 0.005) # 5ms
        out[pos:pos+duration] = np.random.uniform(-0.5, 0.5, min(duration, chunk_len-pos))
    return out

# Ograniczenie pasma częstotliwości (efekt "telefonu")
elif np.random.rand() < 0.12:
    nyquist = self.rate / 2
    low = np.random.uniform(300, 500) / nyquist
    high = np.random.uniform(3000, 4000) / nyquist
    b, a = butter(2, [low, high], btype='band')
```

```

    return lfilter(b, a, samples)
# Brak efektu
else:
    return samples

```



Rysunek 17: Ikona efektu “Online Classroom”

5.11 Ring Modulation

Modulacja pierścieniowa w programie polega na mnożeniu sygnału wejściowego przez falę sinusoidalną o ustalonej częstotliwości (tutaj 300 Hz). W wyniku tego powstają nowe składowe widmowe: suma i różnica częstotliwości sygnału oraz nośnej. Efekt daje charakterystyczne metaliczne, dzwonowe brzmienie, a jego celem jest transformacja mowy w nietypowe, eksperymentalne barwy.



Rysunek 18: Ikona efektu ‘Ring Modulation’

5.12 Stutter

Stutter to efekt polegający na powtarzaniu krótkiego fragmentu sygnału w sposób zapętłony. W praktyce wybierany jest ułamek ramki (1/8 długości), a następnie powielany i docinany do długości całego bloku. W rezultacie otrzymujemy zglitchowany efekt “jąkania się”.



Rysunek 19: Ikona efektu ‘Stutter’

6 Wizualizacja sygnału

Moduł wizualizacji pracuje w wątku GUI i otrzymuje próbki w formacie float32 znormalizowane do przedziału $[-1, 1]$, pochodzące z toru wyjściowego (po efektach). Dla przebiegu czasowego utrzymywany jest cykliczny bufor o długości 0,25 s. Każda nowa porcja danych dosuwana jest na jego koniec, a najstarsze próbki są odrzucane. Oś czasu jest stała i ustawiona względem “teraz”: od wartości ujemnych do zera, co pozwala stale oglądać ostatni wycinek sygnału bez zmiany skali ani przewijania. Dla płynności rysowania stosowana jest lekka decymacja (co druga próbka trafia na wykres), bez wpływu na czytelność kształtu fali dla częstotliwości mowy.

Spektrogram powstaje z ostatnich 1024 próbek ($NFFT = 1024$) okienkowanych funkcją Hanninga. Z każdej takiej porcji liczona jest transformata Fouriera część dodatnia widma ($rfft$), po czym moduł jest przeliczany na dBFS według

$$20 \cdot \log_{10} |X|, \quad (1)$$

z numeryczną stabilizacją dla bardzo małych wartości. Skala amplitudy jest przycinana do przedziału $(-90, 0)$ dBFS, a kolumna widma dopisywana jest jako ostatnia kolumna obrazu spektrogramu. Macierz spektrogramu ma wymiary $(NFFT/2 + 1) \times 240$ kolumn i przewija się w lewo o jedną kolumnę przy każdym odświeżeniu, dzięki czemu pozioma oś czasu reprezentuje około jedenastu sekund historii ($240 \times 2048/44100 \approx 11.0$ s). Oś częstotliwości sięga do Nyquista (22.05 kHz), a mapa kolorów jest stała i dobrana tak, by zachować kontrast przy ciemnym motywie interfejsu.

Rysowanie wykorzystuje blitowanie Matplotlib: po pierwszym pełnym renderze przechowywany jest obraz tła każdej osi i odświeżana jest wyłącznie zmieniająca się warstwa (linia przebiegu lub raster spektrogramu). Pozwala to uzyskać płynne odświeżanie. W momencie zmiany rozmiaru okna tło jest ponownie próbkowane, aby uniknąć artefaktów. Interwał odświeżania jest zsynchronizowany z rozmiarem porcji audio (2048 próbek przy 44.1 kHz, czyli około 46 ms), dzięki czemu rytm wizualizacji odpowiada rytmowi przetwarzania w wątku audio i nie wprowadza niepotrzebnych kolejek zdarzeń.

7 Obsługa - Podsumowanie i praktyczne wskazówki

Obsługa aplikacji została uproszczona tak, aby była intuicyjna dla wszystkich użytkowników niezależnie od umiejętności technicznych. Po uruchomieniu programu mikrofon jest aktywny, a sygnał trafia na wyjście bez efektów. Wybór efektu odbywa się przez kliknięcie odpowiedniego przycisku w zakładce *Effects*; aktywny przycisk zmienia kolor, co jednoznacznie wskazuje stan. Kliknięcie go ponownie wyłącza efekt i przywraca czysty sygnał. Zaleca się korzystanie z dobrych słuchawek, aby uniknąć sprzężenia zwrotnego, które może wystąpić przy pracy na głośnikach.

Przycisk mikrofonu w dolnym prawym rogu pozwala szybko wyciszyć wejście, co jest przydatne przy testach lub w sytuacjach. Zakładka *Plots* umożliwia obserwację sygnału w czasie rzeczywistym — waveform przydatny jest do oceny dynamiki i kształtu przebiegu, a spektrogram do analizy zmian częstotliwościowych. Funkcja nagrywania pozwala zachować próbki efektów; po zatrzymaniu nagrania pojawia się okno wyboru pliku i formatu.

8 Struktura projektu (pliki kluczowe) i rozwiązywanie problemów

Projekt składa się z kilku podstawowych plików, które odpowiadają za różne warstwy działania aplikacji.

- **Voice_Changer.py** – główny plik aplikacji; implementacja logiki GUI (MainWindow), wątku audio (AudioWorker) oraz algorytmów DSP (VoiceChanger).
- **main_app_window_cps.ui** – plik interfejsu użytkownika wygenerowany w Qt Designerze; definiuje układ zakładek, przycisków i obszaru wizualizacji.
- **app_stylesheet.py** – moduł ustawiający paletę kolorów i podstawowe reguły wyglądu interfejsu.
- **darkgreen_palette.qss** – plik arkusza stylów Qt (QSS) zawierający szczegółowe reguły dla przycisków, pól edycyjnych i zakładek.
- **CPS_resources_from_qt.qrc** / **.py** – plik zasobów z ikonami efektów (SVG), skompilowany do postaci Pythona; zapewnia poprawne wbudowanie grafik w aplikacji.

Jeśli aplikacja w wersji skompilowanej (.exe) nie uruchamia się na danym urządzeniu, projekt można uruchomić bezpośrednio z kodu źródłowego w środowisku Python, który również jest dostarczany dla użytkownika.

Aby to zrobić, należy:

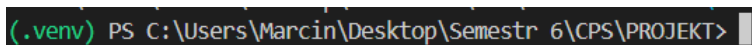
1. Zainstalować odpowiednie środowisko Python

Zalecana jest wersja 3.10–3.12. W systemie powinien być aktywny mikrofon oraz urządzenie wyjściowe audio (słuchawki lub głośniki).

- ### 2. Dobrą praktyką jest korzystanie z tzw. środowiska wirtualnego (venv), które izoluje biblioteki potrzebne dla aplikacji od reszty systemu. W systemie Windows będąc w katalogu projektu w terminalu należy uruchomić kolejno komendy:

```
python -m venv .venv
.venv\Scripts\activate
```

Po aktywacji wiersz poleceń zostanie opatrzony prefiksem (.venv), co oznacza, że biblioteki będą instalowane lokalnie w tym środowisku (Rysunek 20).



```
(.venv) PS C:\Users\Marcin\Desktop\Semestr 6\CPS\PROJEKT>
```

Rysunek 20: Wiersz poleceń z prefiksem “.venv” po poprawnym zainstalowaniu środowiska wirtualnego

3. Zainstalować wymagane biblioteki.

W aktywnym środowisku należy doinstalować biblioteki:

- **PySide6** – interfejs graficzny Qt,
- **PyAudio** – obsługa wejścia/wyjścia audio,
- **numpy, scipy, matplotlib** – obliczenia i wizualizacja,

Instalacja odbywa się poleceniem:

```
pip install PySide6 PyAudio numpy scipy matplotlib
```

4. Skompilować zasoby graficzne.

Plik **CPS_resources_from_qt.qrc** zawierający ikony należy przekonwertować do postaci Pythona poleceniem:

```
pyside6-rcc CPS_resources_from_qt.qrc -o CPS_resources_from_qt.py
```

Zapewnia to poprawne wczytywanie ikon w aplikacji.

5. Uruchomić program.

Po spełnieniu powyższych kroków aplikację uruchamia się poleceniem:

```
python Voice_Changer.py
```

Po wykonaniu poprawnie wszystkich kroków program uruchomi się w identycznej formie jak wersja .exe, korzystając z domyślnego mikrofonu i głośników.

Literatura

[1] Massachusetts Institute of Technology. Pyaudio documentation. URL <https://people.csail.mit.edu/hubert/pyaudio/docs>. Dostęp: 25 sierpnia 2025.

The Qt Company. Qt for Python — PySide6 Documentation. Dostępne online: <https://doc.qt.io/qtforpython/> (dostęp: 30 sierpnia 2025).

The SciPy Community. SciPy Documentation (Release 1.11+). Dostępne online: <https://docs.scipy.org/doc/scipy/> (dostęp: 30 sierpnia 2025).

The SciPy Community. `scipy.signal.lfilter` — Linear filtering. Dostępne online: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.lfilter.html> (dostęp: 30 sierpnia 2025).

The SciPy Community. `scipy.signal.butter` — Butterworth filter design. Dostępne online: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.butter.html> (dostęp: 30 sierpnia 2025).

The SciPy Community. `scipy.signal.fftconvolve` — Convolution using FFT. Dostępne online: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.fftconvolve.html> (dostęp: 30 sierpnia 2025).

The NumPy Community. `numpy.fft.fft` — Discrete Fourier Transform. Dostępne online: <https://numpy.org/doc/stable/reference/generated/numpy.fft.fft.html> (dostęp: 30 sierpnia 2025).

The NumPy Community. `numpy.fft.ifft` — Inverse Discrete Fourier Transform. Dostępne online: <https://numpy.org/doc/stable/reference/generated/numpy.fft.ifft.html> (dostęp: 30 sierpnia 2025).

The NumPy Community. `numpy.fft.rfft` — FFT of real-valued signal. Dostępne online: <https://numpy.org/doc/stable/reference/generated/numpy.fft.rfft.html> (dostęp: 30 sierpnia 2025).